AI/ML Algorithms

Study Guide and Reference

Complete Collection of Algorithms with Definitions and Python Code

Table of Contents

Right-click and select Update Field to refresh page numbers

Table of Contents **2**

1. Search Algorithms **4**

1.1 Breadth First Search (BFS) 4

1.2 Depth First Search (DFS) 4

1.3 A-Star Search Algorithm 4

1.4 Best First Search 5

1.5 AO-Star Search (AND-OR Graph Search) 6

1.6 Branch and Bound 6

2. Classical AI Problems **8**

2.1 Simple Chatbot 8

2.2 Water Jug Problem 8

2.3 Missionaries and Cannibals Problem 9

3. Data Preprocessing and Visualization **11**

3.1 Scatter Plot Visualization 11

3.2 Handling Missing Values (SimpleImputer) 11

3.3 Categorical Encoding (LabelEncoder and OneHotEncoder) 12

3.4 Feature Standardization 12

3.5 Train-Test Split 12

4. Regression Algorithms **14**

4.1 Simple Linear Regression 14

4.2 Multiple Linear Regression 14

4.3 Polynomial Regression 15

4.4 Decision Tree Regression 15

4.5 Support Vector Regression (SVR) 16

5. Classification Algorithms **17**

5.1 Naive Bayes Classifiers 17

5.2 Random Forest 18

5.3 Kernel SVM 18

5.4 Decision Tree Classifier 19

5.5 k-Nearest Neighbors (k-NN) 19

6. Clustering Algorithms **21**

6.1 K-Means Clustering 21

6.2 Elbow Method for Optimal k 21

6.3 Agglomerative Clustering 22

7. Association Rule Mining 23
7.1 Apriori Algorithm 23
7.2 ECLAT Algorithm 23
7.3 FP-Growth Algorithm 24

1. Search Algorithms

Search algorithms are fundamental to AI and are used to explore graphs, trees, and state spaces to find paths or solutions.

1.1 Breadth First Search (BFS)

BFS explores all nodes at the present depth level before moving on to nodes at the next depth level. It uses a queue data structure and guarantees the shortest path in unweighted graphs.

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)

    while queue:
        vertex = queue.popleft()
        print('Visited:', vertex)

        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

1.2 Depth First Search (DFS)

DFS explores as far as possible along each branch before backtracking. It uses recursion (or a stack) and is useful for topological sorting, cycle detection, and maze solving.

```
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()

    visited.add(start)
    print('Visited:', start)

    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
```

1.3 A-Star Search Algorithm

A-Star is an informed search algorithm that uses a heuristic function to estimate the cost from the current node to the goal. It combines the actual cost (g) and heuristic estimate (h) to find the optimal path efficiently.

```
import heapq

def a_star(graph, heuristic, start, goal):
    open_list = []
    heapq.heappush(open_list, (heuristic[start], start))

    g_cost = {start: 0}
    parent = {start: None}

    while open_list:
        f, current = heapq.heappop(open_list)

        if current == goal:
            path = []
            while current:
                path.append(current)
                current = parent[current]
            return path[::-1]

        for neighbor, cost in graph[current]:
            tentative_g = g_cost[current] + cost

            if neighbor not in g_cost or tentative_g < g_cost[neighbor]:
                g_cost[neighbor] = tentative_g
                f_cost = tentative_g + heuristic[neighbor]
                heapq.heappush(open_list, (f_cost, neighbor))
                parent[neighbor] = current

    return None
```

1.4 Best First Search

Best First Search uses only the heuristic function (h) to select the next node to explore. It is a greedy algorithm that always expands the node that appears closest to the goal.

```
import heapq

def best_first_search(graph, heuristic, start, goal):
    visited = set()
    priority_queue = []
    heapq.heappush(priority_queue, (heuristic[start], start))

    while priority_queue:
        h, current = heapq.heappop(priority_queue)

        if current in visited:
            continue

        visited.add(current)

        if current == goal:
            print('Goal reached!')
            return

        for neighbor in graph[current]:
            if neighbor not in visited:
                heapq.heappush(priority_queue, (heuristic[neighbor], neighbor))
```

```
    print('Goal not found')
```

1.5 AO-Star Search (AND-OR Graph Search)

AO-Star is used for searching AND-OR graphs where some nodes require all children to be solved (AND nodes) while others require only one child (OR nodes). It is commonly used in problem reduction scenarios.

```
import math

class Node:
    def __init__(self, name, heuristic):
        self.name = name
        self.h = heuristic
        self.children = []
        self.solved = False

    def add_children(self, child_nodes, cost):
        self.children.append((child_nodes, cost))

def ao_star(node):
    if not node.children:
        node.solved = True
        return node.h

    min_cost = math.inf
    best_children = None

    for child_set, cost in node.children:
        total_cost = cost
        for child in child_set:
            total_cost += ao_star(child)

        if total_cost < min_cost:
            min_cost = total_cost
            best_children = child_set

    node.h = min_cost
    node.solved = True
    return min_cost
```

1.6 Branch and Bound

Branch and Bound is an optimization algorithm that systematically explores the search space by branching into subproblems and using bounds to prune unpromising branches. It guarantees finding the optimal solution.

```
import heapq

def branch_and_bound(graph, start, goal):
    pq = []
    heapq.heappush(pq, (0, [start]))
```

```
    best_cost = float('inf')
    best_path = None

    while pq:
        cost, path = heapq.heappop(pq)
        current = path[-1]

        if cost >= best_cost:
            continue

        if current == goal:
            best_cost = cost
            best_path = path
            continue

        for neighbor, edge_cost in graph[current]:
            if neighbor not in path:
                new_cost = cost + edge_cost
                new_path = path + [neighbor]
                heapq.heappush(pq, (new_cost, new_path))

    return best_path, best_cost
```

2. Classical AI Problems

These are classic problems that demonstrate fundamental AI concepts and search techniques.

2.1 Simple Chatbot

A rule-based chatbot that responds to user inputs based on predefined patterns. It demonstrates natural language processing basics and conditional response generation.

```
def chatbot():
    print('Chatbot: Hello! I am a simple chatbot.')
    print('Chatbot: Type bye to exit.')

    while True:
        user_input = input('You: ').lower()

        if user_input in ['hi', 'hello']:
            print('Chatbot: Hello! How can I help you?')
        elif 'how are you' in user_input:
            print('Chatbot: I am doing great!')
        elif 'your name' in user_input:
            print('Chatbot: You can call me AI Chatbot.')
        elif 'help' in user_input:
            print('Chatbot: I can answer simple questions.')
        elif user_input == 'bye':
            print('Chatbot: Goodbye!')
            break
        else:
            print('Chatbot: Sorry, I do not understand.')
```

2.2 Water Jug Problem

The Water Jug Problem is a classic puzzle where you have two jugs of different capacities and need to measure a specific amount of water using only fill, empty, and pour operations.

```
def water_jug_problem():
    A, B = 0, 0
    capA, capB = 4, 3
    target = 2

    print('WATER JUG PROBLEM')
    print(f'Jug A: {capA}L, Jug B: {capB}L, Target: {target}L')

    B = capB
    print(f'Step 1: Fill B -> A={A}, B={B}')

    pour = min(B, capA - A)
    A += pour
    B -= pour
    print(f'Step 2: Pour B to A -> A={A}, B={B}')

    B = capB
    print(f'Step 3: Fill B -> A={A}, B={B}')
```

```
    pour = min(B, capA - A)
    A += pour
    B -= pour
    print(f'Step 4: Pour B to A -> A={A}, B={B}')

    if B == target:
        print(f'Target achieved: {target}L in Jug B')
```

2.3 Missionaries and Cannibals Problem

A classic river-crossing puzzle where 3 missionaries and 3 cannibals must cross a river using a boat that holds at most 2 people, with the constraint that missionaries cannot be outnumbered on either bank.

```
from collections import deque

def is_valid(state):
    M_left, C_left, boat = state
    M_right = 3 - M_left
    C_right = 3 - C_left

    if M_left < 0 or C_left < 0 or M_left > 3 or C_left > 3:
        return False

    if M_left > 0 and C_left > M_left:
        return False
    if M_right > 0 and C_right > M_right:
        return False

    return True

def get_next_states(state):
    M_left, C_left, boat = state
    moves = [(2,0), (0,2), (1,1), (1,0), (0,1)]
    next_states = []

    for m, c in moves:
        if boat == 0:
            new_state = (M_left - m, C_left - c, 1)
        else:
            new_state = (M_left + m, C_left + c, 0)

        if is_valid(new_state):
            next_states.append(new_state)

    return next_states

def missionaries_cannibals():
    start = (3, 3, 0)
    goal = (0, 0, 1)

    queue = deque([(start, [])])
    visited = set()

    while queue:
        state, path = queue.popleft()
```

```
        if state == goal:
            print('Solution found!')
            for i, s in enumerate(path + [state]):
                M, C, b = s
                side = 'Left' if b==0 else 'Right'
                print(f'Step {i}: M={M}, C={C}, Boat={side}')
            return

        if state not in visited:
            visited.add(state)
            for next_state in get_next_states(state):
                queue.append((next_state, path + [state]))

    print('No solution found')
```

3. Data Preprocessing and Visualization

Data preprocessing is essential for preparing raw data for machine learning models. These techniques ensure data quality and compatibility with algorithms.

3.1 Scatter Plot Visualization

Scatter plots visualize the relationship between two numerical variables. They are useful for identifying patterns, clusters, and outliers in data.

```
import pandas as pd
import matplotlib.pyplot as plt

iris_df = pd.read_csv('./iris.csv')
colors = {'setosa':'red', 'versicolor':'green', 'virginica':'blue'}

for species in iris_df['species'].unique():
    subset = iris_df[iris_df['species'] == species]
    plt.scatter(subset['sepal_length'], subset['sepal_width'],
        c=colors[species], label=species)

plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.title('Scatter Plot: Sepal Length vs Sepal Width')
plt.legend()
plt.show()
```

3.2 Handling Missing Values (SimpleImputer)

Missing data can be handled by imputation - replacing missing values with statistical measures like mean, median, or mode. SimpleImputer from sklearn provides easy imputation strategies.

```
import pandas as pd
from sklearn.impute import SimpleImputer

iris_df = pd.read_csv('./iris.csv')

print('Null values before:')
print(iris_df.isnull().sum())

imputer = SimpleImputer(strategy='mean')
numerical_cols = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
iris_df[numerical_cols] = imputer.fit_transform(iris_df[numerical_cols])

print('Null values after:')
print(iris_df.isnull().sum())
```

3.3 Categorical Encoding (LabelEncoder and OneHotEncoder)

Machine learning models require numerical input. LabelEncoder converts categories to integers (0, 1, 2...), while OneHotEncoder creates binary columns for each category to avoid ordinal assumptions.

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

data = pd.read_csv('./iris.csv')

# Label Encoding
le = LabelEncoder()
data['species_label'] = le.fit_transform(data['species'])
print('Label Encoded:')
print(data[['species', 'species_label']].head())

# One-Hot Encoding
ohe = OneHotEncoder(sparse_output=False)
encoded = ohe.fit_transform(data[['species']])
encoded_df = pd.DataFrame(encoded, columns=ohe.get_feature_names_out(['species']))
print('One-Hot Encoded:')
print(encoded_df.head())
```

3.4 Feature Standardization

Standardization (Z-score normalization) transforms features to have zero mean and unit variance. This is crucial for algorithms sensitive to feature scales like SVM, k-NN, and neural networks.

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

data = pd.read_csv('./Salary_Data.csv')

scaler = StandardScaler()
data[['Years of Experience', 'Salary']] = scaler.fit_transform(
    data[['Years of Experience', 'Salary']]
)

print(data.head())
```

3.5 Train-Test Split

Splitting data into training and testing sets is essential for evaluating model performance. The training set is used to train the model, while the test set evaluates its generalization ability.

```
import pandas as pd
from sklearn.model_selection import train_test_split
```

```
data = pd.read_csv('./iris.csv')

X = data[['sepal_length', 'sepal_width', 'petal_length', 'petal_width']]
y = data['species']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=10
)

print(f'Training data shape: {X_train.shape}')
print(f'Testing data shape: {X_test.shape}')
```

4. Regression Algorithms

Regression algorithms predict continuous numerical values. They are used for forecasting, trend analysis, and relationship modeling.

4.1 Simple Linear Regression

Simple Linear Regression models the relationship between a single independent variable and a dependent variable using a straight line. It finds the best-fit line that minimizes the sum of squared errors.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

data = pd.read_csv('./house_data.csv')

X = data[['sqft_living']]
y = data['price']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(f'Coefficient: {model.coef_[0]}')
print(f'Intercept: {model.intercept_}')
print(f'MSE: {mean_squared_error(y_test, y_pred)}')
print(f'R2 Score: {r2_score(y_test, y_pred)}')
```

4.2 Multiple Linear Regression

Multiple Linear Regression extends simple regression to use multiple independent variables. It models the linear relationship between several predictors and the target variable.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

data = pd.read_csv('./50_Startups.csv')
data = pd.get_dummies(data, drop_first=True)

X = data.drop('Profit', axis=1)
y = data['Profit']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(f'R2 Score: {r2_score(y_test, y_pred)}')

# Adjusted R2
n = X_test.shape[0]
p = X_test.shape[1]
adj_r2 = 1 - ((1 - r2_score(y_test, y_pred)) * (n - 1) / (n - p - 1))
print(f'Adjusted R2: {adj_r2}')
```

4.3 Polynomial Regression

Polynomial Regression models non-linear relationships by adding polynomial terms (squared, cubed, etc.) to the features. It can capture curved patterns that linear regression cannot.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

data = pd.read_csv('./Position_Salaries.csv')

X = data[['Level']]
y = data['Salary']

poly = PolynomialFeatures(degree=4)
X_poly = poly.fit_transform(X)

model = LinearRegression()
model.fit(X_poly, y)

# Predict for level 6.5
level = pd.DataFrame([[6.5]], columns=['Level'])
level_poly = poly.transform(level)
salary_pred = model.predict(level_poly)
print(f'Predicted Salary: {salary_pred[0]}')
```

4.4 Decision Tree Regression

Decision Tree Regression splits the data into regions based on feature values and predicts the average target value in each region. It can capture complex non-linear patterns without feature engineering.

```
import pandas as pd
from sklearn.tree import DecisionTreeRegressor, export_text
```

```
data = pd.read_csv('./Position_Salaries.csv')

X = data[['Level']]
y = data['Salary']

model = DecisionTreeRegressor(random_state=0)
model.fit(X, y)

# Prediction
level = pd.DataFrame([[6.5]], columns=['Level'])
print(f'Prediction for Level 6.5: {model.predict(level)}')

# Tree structure
print(export_text(model, feature_names=['Level']))
```

4.5 Support Vector Regression (SVR)

SVR uses support vectors to define a margin of tolerance around the predicted line. It is effective for high-dimensional data and can use different kernel functions for non-linear relationships.

```
import pandas as pd
import numpy as np
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score

data = pd.read_csv('./Position_Salaries.csv')

X = data[['Level']]
y = data['Salary']

sc_X = StandardScaler()
sc_y = StandardScaler()

X_scaled = sc_X.fit_transform(X)
y_scaled = sc_y.fit_transform(y.values.reshape(-1, 1))

model = SVR(kernel='rbf')
model.fit(X_scaled, y_scaled.ravel())

# Predict
level = pd.DataFrame([[6.5]], columns=['Level'])
level_scaled = sc_X.transform(level)
salary_scaled = model.predict(level_scaled)
salary = sc_y.inverse_transform(salary_scaled.reshape(-1, 1))
print(f'Predicted Salary: {salary}')

# R2 Score
y_pred_scaled = model.predict(X_scaled)
y_pred = sc_y.inverse_transform(y_pred_scaled.reshape(-1, 1))
print(f'R2 Score: {r2_score(y, y_pred)}')
```

5. Classification Algorithms

Classification algorithms predict discrete class labels. They are used for categorization, diagnosis, and decision-making tasks.

5.1 Naive Bayes Classifiers

Naive Bayes is a probabilistic classifier based on Bayes theorem with strong independence assumptions. It is fast, simple, and works well for text classification and spam filtering.

5.1.1 Gaussian Naive Bayes

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = pd.read_csv('Social_Network_Ads.csv')

X = data[['Age', 'EstimatedSalary']]
y = data['Purchased']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0
)

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

nb = GaussianNB()
nb.fit(X_train, y_train)

y_pred = nb.predict(X_test)

print(f'Accuracy: {accuracy_score(y_test, y_pred)}')
print(f'Confusion Matrix:')
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

5.1.2 Multinomial Naive Bayes

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, confusion_matrix

X = np.array([
    [0,0], [0,1], [1,0], [1,1], [2,2], [2,1],
    [0,2], [1,2], [2,0], [0,1], [1,1], [2,2]
])
y = np.array([0,0,0,1,1,1,0,1,1,0,1,1])
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0
)

nb = MultinomialNB()
nb.fit(X_train, y_train)

y_pred = nb.predict(X_test)
print(f'Accuracy: {accuracy_score(y_test, y_pred)}')
```

5.2 Random Forest

Random Forest is an ensemble method that builds multiple decision trees and aggregates their predictions. It reduces overfitting and improves accuracy through bagging and feature randomness.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score

data = pd.read_csv('Social_Network_Ads.csv')

X = data[['Age', 'EstimatedSalary']]
y = data['Purchased']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0
)

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

classifier = RandomForestClassifier(n_estimators=100, random_state=0)
classifier.fit(X_train, y_train)

y_pred = classifier.predict(X_test)
y_prob = classifier.predict_proba(X_test)[:, 1]

print(f'Confusion Matrix:')
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
print(f'ROC AUC Score: {roc_auc_score(y_test, y_prob)}')
```

5.3 Kernel SVM

Support Vector Machine with kernel trick maps data to higher dimensions to find non-linear decision boundaries. The RBF kernel is commonly used for complex classification tasks.

```
from sklearn.svm import SVC
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

svm = SVC(kernel='rbf', probability=True, random_state=0)
svm.fit(X_train, y_train)

y_pred = svm.predict(X_test)
y_prob = svm.predict_proba(X_test)[:, 1]

print(f'Accuracy: {accuracy_score(y_test, y_pred)}')
print(f'Confusion Matrix:')
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

5.4 Decision Tree Classifier

Decision Tree Classifier creates a tree-like model of decisions based on feature values. It is interpretable and can handle both numerical and categorical data.

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

data = pd.read_csv('tennis.csv')

le = LabelEncoder()
for col in data.columns:
    data[col] = le.fit_transform(data[col])

X = data.iloc[:, :-1]
y = data.iloc[:, -1]

dt = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)
dt.fit(X, y)

y_pred = dt.predict(X)

print(f'Accuracy: {accuracy_score(y, y_pred)}')
print(classification_report(y, y_pred))
```

5.5 k-Nearest Neighbors (k-NN)

k-NN classifies data points based on the majority class of their k nearest neighbors. It is a lazy learner that makes no assumptions about the data distribution.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, confusion_matrix

data = pd.read_csv('User_Data.csv')
```

```
X = data[['Age', 'EstimatedSalary']]
y = data['Purchased']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0
)

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

print(f'Accuracy: {accuracy_score(y_test, y_pred)}')
print(f'Precision: {precision_score(y_test, y_pred)}')
print(f'Confusion Matrix:')
print(confusion_matrix(y_test, y_pred))
```

6. Clustering Algorithms

Clustering algorithms group similar data points together without predefined labels. They are used for customer segmentation, anomaly detection, and pattern discovery.

6.1 K-Means Clustering

K-Means partitions data into k clusters by minimizing the within-cluster sum of squares. It iteratively assigns points to the nearest centroid and updates centroids until convergence.

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score
from scipy.stats import mode
import numpy as np

# Create synthetic dataset
X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=1.0, random_state=42)

# Apply K-Means
kmeans = KMeans(n_clusters=4, random_state=42)
y_kmeans = kmeans.fit_predict(X)

# Map clusters to labels
def map_clusters_to_labels(y_true, y_pred):
    labels = np.zeros_like(y_pred)
    for i in np.unique(y_pred):
        mask = (y_pred == i)
        labels[mask] = mode(y_true[mask])[0]
    return labels

y_mapped = map_clusters_to_labels(y_true, y_kmeans)
print(f'Accuracy: {accuracy_score(y_true, y_mapped)}')
print(f'Cluster Centers:')
print(kmeans.cluster_centers_)
```

6.2 Elbow Method for Optimal k

The Elbow Method helps determine the optimal number of clusters by plotting the Within-Cluster Sum of Squares (WCSS) against different k values. The elbow point indicates the optimal k.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

data = pd.read_csv('Mall_Customers.csv')

X = data[['Annual Income (k$)', 'Spending Score (1-100)']].values
```

```
sc = StandardScaler()
X_scaled = sc.fit_transform(X)

wcss = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.plot(range(1, 11), wcss, marker='o')
plt.title('Elbow Method')
plt.xlabel('Number of clusters (k)')
plt.ylabel('WCSS')
plt.show()

# Apply with optimal k=5
optimal_k = 5
kmeans = KMeans(n_clusters=optimal_k, random_state=42)
y_kmeans = kmeans.fit_predict(X_scaled)
```

6.3 Agglomerative Clustering

Agglomerative Clustering is a hierarchical method that starts with each point as its own cluster and iteratively merges the closest pairs until the desired number of clusters is reached.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

# Generate data
X, y_true = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=42)

# Create dendrogram
linked = linkage(X, method='ward')
plt.figure(figsize=(10, 5))
dendrogram(linked)
plt.title('Dendrogram')
plt.show()

# Apply Agglomerative Clustering
agg_cluster = AgglomerativeClustering(n_clusters=3, linkage='ward')
y_pred = agg_cluster.fit_predict(X)

print('Clustering completed')
```

7. Association Rule Mining

Association Rule Mining discovers interesting relationships between variables in large datasets. It is commonly used in market basket analysis to find products frequently purchased together.

7.1 Apriori Algorithm

Apriori finds frequent itemsets and generates association rules. It uses the Apriori principle: if an itemset is frequent, all its subsets must also be frequent.

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder

# Sample transactions
transactions = [
    ['Milk', 'Bread', 'Butter'],
    ['Bread', 'Butter'],
    ['Milk', 'Bread'],
    ['Milk', 'Butter'],
    ['Bread', 'Butter'],
    ['Milk', 'Bread', 'Butter', 'Eggs']
]

# Convert to one-hot encoded format
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_array, columns=te.columns_)

# Find frequent itemsets
frequent_itemsets = apriori(df, min_support=0.5, use_colnames=True)
print('Frequent Itemsets:')
print(frequent_itemsets)

# Generate association rules
rules = association_rules(frequent_itemsets, metric='confidence', min_threshold=0.7)
print('Association Rules:')
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

7.2 ECLAT Algorithm

ECLAT (Equivalence Class Clustering and bottom-up Lattice Traversal) uses a vertical data format (tidsets) and intersects them to find frequent itemsets. It is often faster than Apriori for dense datasets.

```
from itertools import combinations
import pandas as pd

# Sample transactions
transactions = [
    ['Milk', 'Bread', 'Butter'],
    ['Bread', 'Butter'],
```

```
    ['Milk', 'Bread'],
    ['Milk', 'Butter'],
    ['Bread', 'Butter'],
    ['Milk', 'Bread', 'Butter', 'Eggs']
]

# Convert to vertical format
def vertical_format(transactions):
    vertical_data = {}
    for tid, transaction in enumerate(transactions):
        for item in transaction:
            if item not in vertical_data:
                vertical_data[item] = set()
            vertical_data[item].add(tid)
    return vertical_data

vertical_data = vertical_format(transactions)

# ECLAT algorithm
def eclat(prefix, items, min_support, freq_itemsets):
    while items:
        item, tidset = items.pop()
        support = len(tidset)

        if support >= min_support:
            new_prefix = prefix + [item]
            freq_itemsets.append((new_prefix, support))

            new_items = []
            for other_item, other_tidset in items:
                intersection = tidset & other_tidset
                if len(intersection) >= min_support:
                    new_items.append((other_item, intersection))

            eclat(new_prefix, new_items, min_support, freq_itemsets)

min_support = 3
freq_itemsets = []
items = list(vertical_data.items())
eclat([], items, min_support, freq_itemsets)

print('Frequent Itemsets:')
for itemset, support in freq_itemsets:
    print(f'{itemset}: Support = {support}')
```

7.3 FP-Growth Algorithm

FP-Growth uses a compressed FP-Tree structure to store transaction data and mines frequent itemsets without candidate generation. It is significantly faster than Apriori for large datasets.

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import fpgrowth, association_rules

# Sample transactions
transactions = [
    ['Milk', 'Bread', 'Butter'],
```

```
    ['Bread', 'Butter'],
    ['Milk', 'Bread'],
    ['Milk', 'Butter'],
    ['Bread', 'Butter'],
    ['Milk', 'Bread', 'Butter', 'Eggs']
]

# Convert to one-hot encoded format
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_array, columns=te.columns_)

# FP-Growth algorithm
frequent_itemsets = fpgrowth(df, min_support=0.5, use_colnames=True)
print('Frequent Itemsets:')
print(frequent_itemsets)

# Generate association rules
rules = association_rules(frequent_itemsets, metric='confidence', min_threshold=0.7)
print('Association Rules:')
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```